

Architettura dei Calcolatori

Primo appello – 14 giugno 2022 – 1h45

1. [6 pt] Usando le tabelle fornite in appendice si decodifichi il seguente binario RISC-V

0x0014	0x00052483
0x0018	0x00990933
0x001C	0x00450513
0x0020	0x00140413
0x0024	0xFEB448E7

Laddove comparissero istruzioni di alterazione del control flow risalire al punto preciso del salto indicando una label al posto dell'immediato (si sfruttino gli indirizzi a cui risiedono le istruzioni, indicati a sinistra)

```
LOOP:
    lw    s1, 0(a0)
    add   s2, s2, s1
    addi  a0, a0, 4
    addi  s0, s0, 1
    blt   s0, a1, LOOP
```

```
LOOP:
    lw    x9, 0(x10)
    add   x18, x18, x9
    addi  x10, x10, 4
    addi  x8, x8, 1
    blt   x8, x11, LOOP
L'immediato vale 0xFF8 senza Lshift (1pt) e 0xFF0 con Lshift (2pt)
```

2. [7 pt] Scrivere una funzione assembly RISC-V **mystrcpy** che prenda in ingresso una stringa destinazione **dest**, una stringa sorgente **src**, un numero di bytes **b** e una direzione **d**. La funzione deve copiare i primi **b** caratteri della stringa **src** sulla stringa **dst**, terminandola col terminatore **'\0'**. Se il numero di bytes **b** indicato dovesse essere più grande del numero di caratteri effettivamente presenti nella stringa **src** quest'ultima verrà interamente copiata nella stringa **dst**. Se la direzione **d** è pari a 0 la copia deve essere diretta, se è pari a 1 deve essere invertita.

Es.

```
char x[10] = "distintivo";
char y[10];
mystrcpy(y, x, 4, 0) produce y = "dist" = {'d', 'i', 's', 't', '\0'}
mystrcpy(y, x, 4, 1) produce y = "tsid" = {'t', 's', 'i', 'd', '\0'}
```

Gestire correttamente lo stack come da calling convention, così come il passaggio dei parametri di ingresso e di uscita sui registri appropriati. Scrivere la funzione main che invoca **mystrcpy**, mostrando anche l'allocazione degli array sul segmento dati statico del file ELF.

MAIN [1,5 pt] – direct copy [2,5 pt] – reverse copy [3 pt]

SOLUZIONE // a0 = dest; a1 = src; a2 = b; a3 = d;

mystrcpy:

```
    addi sp,sp,-24      // adjust stack for 1 dw
    sd    s0,0(sp)      // push s0
    sd    s1,8(sp)      // push s1
    sd    s2,16(sp)     // push s2

    li    s2,0          // i = 0
    bne   a3,zero,L2    // if d != 0 revert copy

L1:  add   s0,s2,a1      // s0 = addr of src[i]
     lbu   s0,0(s0)     // s0 = src[i]
     add   s1,s2,a0      // s1 = addr of dest[i]
     sb    s0,0(s1)     // dest[i] = src[i]
     beq   s0,zero,L4    // if src[i] == 0 then exit
     addi  s2,s2,1       // i = i + 1
     j     L1           // next iteration of loop

L2:  // first determine the length of the SRC string
     add   s0,s2,a1      // s0 = addr of src[i]
     lbu   s0,0(s0)     // s0 = src[i]
     beq   s0,zero,L22   // if src[i] == 0 then exit
     addi  s2,s2,1       // i = i + 1
     j     L2           // next iteration of loop

L22: // s2 now contains the length of the string
     blt   s2,a2,L23     // if (s2<b) goto L23 (consider strlen)
     mov   s2,a2         // s2 = b (consider B)

L23: // first handle termination character
     add   s1,s2,a0      // s1 = addr of dest[i]
     sb    zero,0(s1)    // dest[strlen(STR)] = \0
     addi  s2,s2,-1      // i = i - 1
     mv    a2, s2        // adjust B
     li    s2, 0         // clear i (s2)

L3:  add   s0,s2,a1      // s0 = addr of src[i]
     lbu   s0,0(s0)     // s0 = src[i]
     add   s1,a2,-1      // s1 = b - 1
     sub   s1,s1,s2      // s1 = b - 1 - i
     add   s1, s1,a0      // s1 = addr of dest[b-1-i]
     sb    s0,0(s1)     // dest[i] = src[i]
     beq   s2,zero,L4    // if src[i] == 0 then exit
     addi  s2,s2,-1      // i = i - 1
     j     L23          // next iteration of loop

L4:  ld    s0,0(sp)      // restore saved s2
     ld    s1,8(sp)      // restore saved s2
     ld    s2,16(sp)     // restore saved s2
     addi  sp,sp,24      // pop 1 dw from stack
     ret                    // and return
```

3. Si consideri una cache che usa 5 bit per il campo offset e 3 bit per il campo index. Ipotizzando di aver già letto l'indirizzo 0x0000, quali affermazioni sono vere (spiegare il ragionamento per arrivare alla risposta)?
- Se eseguire letture agli indirizzi 0x0114, 0x0308, 0x1005 produce tutte conflict miss allora la cache è direct mapped
 - Se la stessa sequenza di letture del punto a) produce tutte cold cache miss allora la cache è 4-way set-associative
 - Se la stessa sequenza di letture del punto a) produce una cold cache miss e due conflict miss allora la cache è 2-way set-associative
 - Se la cache è 4-way set-associative la sua dimensione è 1KB

La cache usa 3 bit per il campo index, quindi avrà $2^3=8$ linee, se direct mapped, oppure 8 sets, se set-associative. Ogni linea è grande $2^5=32$ bytes.

- a) Se la cache è direct-mapped, la sequenza di letture produce

Address	TAG	INDEX	OFFSET		HIT/MISS
0x0000	00...00	000	0 0000	nessun dato in cache	cold cache miss
0x0114	00...00	000	1 0100	stesso INDEX, stesso TAG	HIT
0x0308	00...11	000	0 1000	stesso INDEX, TAG diverso	conflict miss
0x1005	00...01 0000	000	0 0101	stesso INDEX, TAG diverso	conflict miss

La prima lettura a 0x0114 è una hit, quindi il punto a) è **FALSO**

- Se la cache è 4-way set associative avrà fino a quattro indirizzi con INDEX uguale e TAG diverso dentro lo stesso set. In questo caso la sequenza di letture del punto a) produrrebbe una HIT e due cold cache miss. La risposta è comunque **FALSA**.
- Come per il punto b), con una cache 2-way set-associative avrà fino a due indirizzi con INDEX uguale e TAG diverso dentro lo stesso set. La sequenza di letture al punto a) produce una HIT, una cold cache miss e una conflict miss. La risposta è comunque **FALSA**.
- In una cache 4-way set-associative ogni set ha 4 ways (ovvero ospita l'equivalente di 4 linee). Dato che ogni linea è grande 32 bytes, ogni set è grande $32*4=128$ bytes. Con 8 sets nella cache, la sua dimensione totale sarà $128*8=1024$ bytes=1KB. La risposta è **VERA**.

4. [5 pt] Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e con dynamic branch predictor a **singolo bit** inizialmente settato a BRANCH TAKEN.

```

li    t2,3 // UB
li    t0,0 // i = 0
loop_i
li    t1,0 // j = 0
loop_j
addi  t1,t1,1
blt   t1,t2,loop_j
addi  t0,t0,1
blt   t0,t2,loop_i

jr    ra

```

a) Si calcoli la misprediction rate [2,5 pt].

b) Sapendo che:

- per ogni predizione corretta un'istruzione di tipo branch impiega 2 cicli ad eseguire, mentre in caso di misprediction ne impiega 8
- le istruzioni aritmetico-logiche impiegano 2 cicli a eseguire
- il salto incondizionato impiega 2 cicli ad eseguire

Si calcoli il CPI medio per l'esecuzione del programma [2,5 pt]

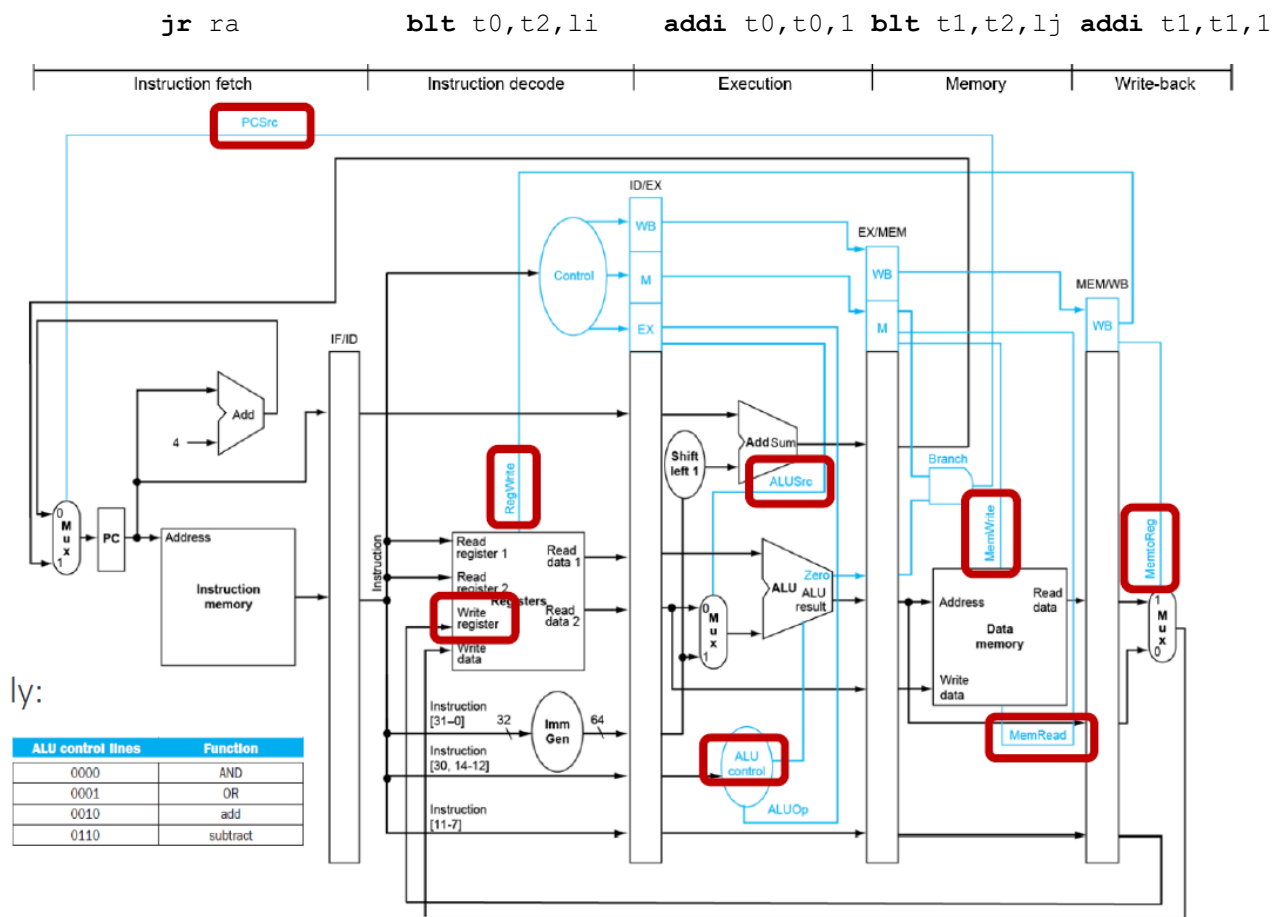
	istanza di istruzione	registri e condizioni	esito salto	2bit pred		1bit pred		miss#
1)	li t0, 0	t0 = 0						
2)	li t2, 3	t2 = 3						
3)	li t1, 0	t1 = 0						
4)	addi t1, t1, 1	t1 = 1						
5)	blt t1, t2, loop_j	t1<t2 → 1<3	T	Ts	H	T	H	[1]
6)	addi t1, t1, 1	t1 = 2						
7)	blt t1, t2, loop_j	t1<t2 → 2<3	T	Ts	H	T	H	[2]
8)	addi t1, t1, 1	t1 = 3						
9)	blt t1, t2, loop_j	t1<t2 → 3<3	N	Ts	M	T	M	[3]
10)	addi t0, t0, 1	t0 = 1						
11)	blt t0, t2, loop_i	t0<t2 → 1<3	T	Tm	H	N	M	[4]
12)	li t1, 0	t1 = 0						
13)	addi t1, t1, 1	t1 = 1						
14)	blt t1, t2, loop_j	t1<t2 → 1<3	T	Ts	H	T	H	[5]
15)	addi t1, t1, 1	t1 = 2						
16)	blt t1, t2, loop_j	t1<t2 → 2<3	T	Ts	H	T	H	[6]
17)	addi t1, t1, 1	t1 = 3						
18)	blt t1, t2, loop_j	t1<t2 → 3<3	N	Ts	M	T	M	[7]
19)	addi t0, t0, 1	t0 = 2						
20)	blt t0, t2, loop_i	t0<t2 → 2<3	T	Tm	H	N	M	[8]
21)	li t1, 0	t1 = 0						
22)	addi t1, t1, 1	t1 = 1						
23)	blt t1, t2, loop_j	t1<t2 → 1<3	T	Ts	H	T	H	[9]
24)	addi t1, t1, 1	t1 = 2						
25)	blt t1, t2, loop_j	t1<t2 → 2<3	T	Ts	H	T	H	[10]
26)	addi t1, t1, 1	t1 = 3						
27)	blt t1, t2, loop_j	t1<t2 → 3<3	N	Ts	M	T	M	[11]
28)	addi t0, t0, 1	t0 = 3						
29)	blt t0, t2, loop_i	t0<t2 → 3<3	N	Tm	M	N	H	[12]
30)	j r ra							

Su 30 istruzioni totali 12 sono branch, 17 aritmetiche (CPI=2) e 1 salto incondizionato (CPI=2).
Per il singolo predittore la misprediction rate è 5/12. Quindi il CPI medio per il programma è

$$CPI = \frac{17 I_{ALU} * 2 \frac{cicli}{I_{ALU}} + 1 I_{JMP} * 2 \frac{cicli}{I_{JMP}} + 12 I_{BR} \left(\frac{5 mis * 8 \frac{cicli}{mis} + 7 pr_{ok} * 2 \frac{cicli}{pr_{ok}}}{12} \right)}{30 I}$$

$$= \frac{34 + 2 + 12 \left(\frac{40 + 14}{12} \right)}{30} = \frac{90}{30} = 3$$

5. [4 pt] Si consideri il programma assembly RISC-V dell'esercizio 4. Considerando il diagramma della pipeline RISC-V single-cycle indicare quanto valgono i segnali di controllo cerchiati in figura quando l'ultima istruzione, la **jr ra**, si trova in fase di fetch).



Osservando il codice dell'esercizio 4 si può riempire la pipeline come indicato in cima alla figura. Da qui si possono stabilire i valori dei segnali di controllo:

PCSrc = 0: In fase di MEM (dove viene generato il segnale) c'è una BLT. Sappiamo però che il flag ZERO è nullo, quindi PCSrc = 0

RegWrite = 1: nella fase di WB c'è una addi, che scrive sul register file

Write register = 6: la addi nella fase di WB scrive su t1, ovvero x6

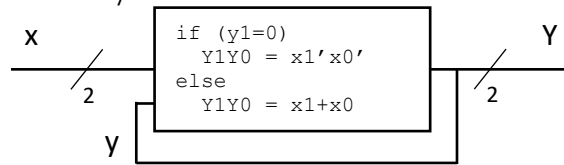
ALUSrc = 1: Nella fase di EXE c'è una addi, il cui secondo operando è un immediato. Il valore dell'immediato viene dalla immediate generation unit, e non dal register file.

ALUctrl = 0010: La ALU è settata per eseguire una ADD

MemWrite = MemRead = 0: Nella fase di MEM c'è una blt, che non legge né scrive la memoria.

MemtoReg = 0: Il risultato scritto sul register file viene dalla ALU, non dalla memoria.

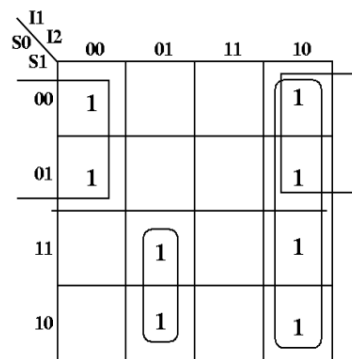
6. [7 pt] Progettare una rete logica sequenziale che riceva in ingresso un segnale X a due bit e fornisca in uscita un segnale Y a due bit. Il valore di Y è pari alla somma dei singoli bit che compongono x, tranne in un caso. Se l'uscita precedente (y) aveva il bit più significativo pari a 0 allora l'uscita attuale (Y) ha i bit equivalenti al complemento dei corrispondenti bit dell'ingresso (x). All'accensione o al reset y=00.



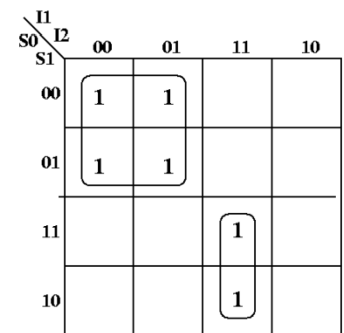
- [1,5 pt] disegnare la macchina a stati finiti. Dire se si sceglie Mealy o Moore, giustificando la scelta;
- [2 pt] scrivere la tabella di verità per uscite e stato futuro;
- [2 pt] trovare le forme minime tramite mappe di Karnaugh;
- [1,5 pt] disegnare il circuito finale.

Tabelle di verità per NextState

S0	S1	I1	I2	S0*	S1*
0	0	0	0	1	1
0	0	0	1	1	0
0	0	1	0	0	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	1	0



$$S1^* = \sim S0 \cdot \sim I2 + I1 \cdot I2 + S0 \cdot \sim I1 \cdot I2$$



$$S0^* = \sim S0 \cdot \sim I1 + I1 \cdot I2 \cdot S0$$

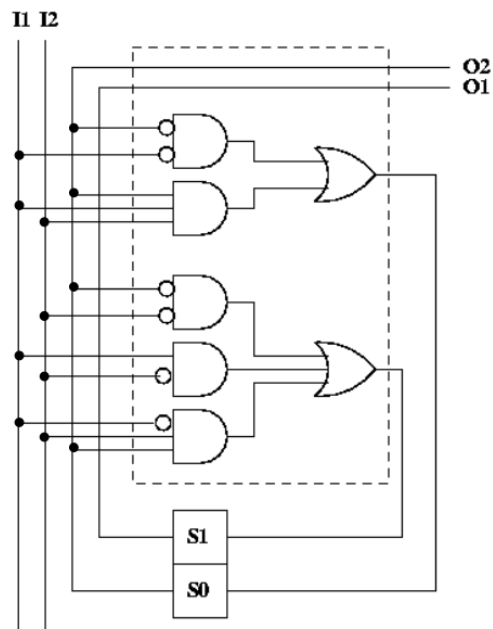
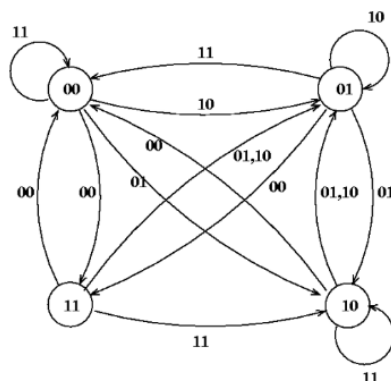
Circuito finale

$$S1^* = \sim S0 \cdot \sim I2 + I1 \cdot I2 + S0 \cdot \sim I1 \cdot I2$$

$$S0^* = \sim S0 \cdot \sim I1 + I1 \cdot I2 \cdot S0$$

$$O1 = S0$$

$$O2 = S1$$



Appendice: registri RISC-V

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value